FLAIR (clean version + analysis)

Cleaned-up copy of the FLAIR Colab notebook (original: https://github.com/conservation-technology-group/FLAIR). Behaviour is unchanged; this version only removes dead code, moves every path and setting into **one config cell**, and fails early with a clear message when something is wrong, so that **Runtime > Run all** works start to end.

Before you run: use a GPU runtime, and check the paths/settings in the *Configuration* cell.

What is different from `FLAIR_efficient`: the pipeline's results are unchanged. It additionally counts the work done at each stage, keeps a copy of *every* track before FLAIR's final filter deletes the unconfirmed ones, and ends with an *Analysis* section (funnel and compute, a per-track feature table, and how CLIP scores separate candidates on the kept track from the rest). Results go to `runs/<RUN_NAME>/` (default run name `analysis_dev`, so your `dev` run is not touched).

Where things go

- Fast local scratch (lost on disconnect): `WORK_DIR` - frames, intermediate masks, overlay frames.
- Persistent results: `PROJECT_DIR/runs/<RUN_NAME>/` on Drive - masks (zipped), boxes, tracked video, `run_info.json` (config, timings, environment).
- Nothing outside `PROJECT_DIR/runs/<RUN_NAME>/` is ever written to or deleted on Drive. The original notebook's outputs are only *read*, for the optional regression check at the end.

```
try:
    from google.colab import drive
    drive.mount('/content/drive')
except ImportError:
    print("Not running on Colab - skipping Drive mount")
```

```
Mounted at /content/drive
```

Configuration

```
import json
import os
import shutil
from pathlib import Path

# ============================== EDIT THIS CELL ==============================
# --- Locations ---------------------------------------------------------------
PROJECT_DIR  = "/content/drive/MyDrive/DS340WCourseProject"                    # persistent (Drive)
SOURCE_VIDEO = f"{PROJECT_DIR}/FLAIR_code/FLAIR/example_video_blacktip.mp4"    # input video (on Drive)
WORK_DIR     = "/content/flair_work"                                           # fast local scratch

RUN_NAME        = "analysis_dev"    # results go to PROJECT_DIR/runs/<RUN_NAME>/. Use a new name for rur
OVERWRITE       = True     # True: replace an existing Drive run folder of this name. False: stop if it
FORCE_REEXTRACT = False    # True: re-extract video frames even if they are already cached locally

# Optional: pruned masks from the original notebook, used (read-only) for the regression check at the er
BASELINE_PRUNED_DIR = f"{PROJECT_DIR}/video_segments_example_video_pruned"
REGRESSION_MIN_DICE = 0.95

# --- Video clip to process ---------------------------------------------------
CLIP_START = "00:00:00"
CLIP_END   = "00:00:08"    # None = whole video

# --- SAM2 --------------------------------------------------------------------
MODEL_CFG     = "sam2_hiera_l.yaml"
VOS_OPTIMIZED = False      # can be True on an appropriate GPU
_LEGACY_CKPT  = "/content/drive/MyDrive/checkpoints/sam2_hiera_large.pt"   # where the original notebook
CHECKPOINT    = _LEGACY_CKPT if os.path.exists(_LEGACY_CKPT) else f"{PROJECT_DIR}/checkpoints/sam2_hiera

MASK_GENERATOR_PARAMS = dict(
```

```
    points_per_side=32,
    points_per_batch=32,
    pred_iou_thresh=0.7,
    stability_score_thresh=0.92,
    stability_score_offset=0.7,
    crop_n_layers=1,
    box_nms_thresh=0.3,
    crop_nms_thresh=0.3,
    crop_n_points_downscale_factor=2,
    min_mask_region_area=0,
    use_m2m=False,
)

# --- FLAIR parameters ------------------------------------------------------
RUN_EVERY_N_FRAMES = 30    # run automatic mask generation every N frames (typically the video fps)
MIN_MASK_LENGTH    = 75    # minimum mask bbox width/height in pixels (calibrate from the grid plot belo
MAX_MASK_LENGTH    = 300   # maximum mask bbox width/height in pixels (calibrate from the grid plot belo
BBOX_BUFFER        = 25    # padding around bounding boxes
WINDOW_LENGTH      = 3     # window length for SAM2 video propagation

# --- CLIP ------------------------------------------------------------------
CLIP_CONFIDENCE = 0.95     # confidence threshold for CLIP filtering
CLIP_PROMPTS = [
    "A drone image of white shape on surface of the ocean",
    "A clear zoomed-in drone image of a complete dark nurse shark in the ocean with distinct pectoral fi
    "A drone image of the ocean with no distinct sharks",
    "A drone image of a rock under the surface of the ocean with no sharks",
    "A drone image of sand in the ocean",
    "A drone image of rocks on land",
]
CLIP_CORRECT = [1]         # indices of the prompts above that describe the object you want to keep

# --- Mask review / pruning -------------------------------------------------
N_PER_MASK    = 9          # images per track in the review grid (3x3)
RESIZE_FACTOR = 0.3        # shrink review images to fit
KEEP_KEYS     = [2]        # track IDs that are true positives (read them off the review grid, then edit

# --- Analysis (read-only; does not change the pipeline's results) ---------------------------
TRACK_CLIP_SAMPLES  = 5     # frames per track that are re-scored with CLIP in the track table (0 = skip
CANDIDATE_LABEL_IOU = 0.5   # a CLIP candidate counts as 'on the kept track' if it overlaps that track's
# ============================== END OF SETTINGS ==============================

# ---- Derived paths and early checks (do not edit) ---------------------------------------
if not RUN_NAME or Path(RUN_NAME).name != RUN_NAME or RUN_NAME in (".", ".."):
    raise ValueError(f"RUN_NAME must be a simple folder name, got {RUN_NAME!r}")
if not os.path.isfile(SOURCE_VIDEO):
    raise FileNotFoundError(
        f"Source video not found: {SOURCE_VIDEO}\nCheck PROJECT_DIR / SOURCE_VIDEO in this cell "
        f"and that Google Drive mounted correctly."
    )

FRAMES_DIR   = f"{WORK_DIR}/frames/{Path(SOURCE_VIDEO).stem}"
RUN_LOCAL    = f"{WORK_DIR}/runs/{RUN_NAME}"
RUN_DRIVE    = f"{PROJECT_DIR}/runs/{RUN_NAME}"
SEGMENTS_DIR = f"{RUN_LOCAL}/video_segments"
PRUNED_DIR   = f"{RUN_LOCAL}/video_segments_pruned"
OVERLAY_DIR  = f"{RUN_LOCAL}/tracked_frames"
OUTPUT_TXT   = f"{RUN_LOCAL}/keep_boxes.txt"
OUTPUT_VIDEO = f"{RUN_LOCAL}/tracked_video.mp4"
SNAPSHOT_DIR = f"{RUN_LOCAL}/video_segments_all_tracks"   # every track, before FLAIR's final filter
ANALYSIS_DIR = f"{RUN_LOCAL}/analysis"

if os.path.exists(RUN_DRIVE) and not OVERWRITE:
    raise FileExistsError(
        f"Run folder already exists: {RUN_DRIVE}\n"
        f"Set OVERWRITE = True to replace it, or choose a new RUN_NAME."
    )

# Local run folder always starts clean (frames are cached separately in FRAMES_DIR).
shutil.rmtree(RUN_LOCAL, ignore_errors=True)
os.makedirs(RUN_LOCAL)
```

```
_CONFIG_KEYS = [
    "PROJECT_DIR", "SOURCE_VIDEO", "WORK_DIR", "RUN_NAME", "OVERWRITE", "CLIP_START", "CLIP_END",
    "MODEL_CFG", "VOS_OPTIMIZED", "CHECKPOINT", "MASK_GENERATOR_PARAMS", "RUN_EVERY_N_FRAMES",
    "MIN_MASK_LENGTH", "MAX_MASK_LENGTH", "BBOX_BUFFER", "WINDOW_LENGTH", "CLIP_CONFIDENCE",
    "CLIP_PROMPTS", "CLIP_CORRECT", "N_PER_MASK", "RESIZE_FACTOR", "KEEP_KEYS",
    "TRACK_CLIP_SAMPLES", "CANDIDATE_LABEL_IOU",
]
CONFIG = {k: globals()[k] for k in _CONFIG_KEYS}

print(f"Source video : {SOURCE_VIDEO}")
print(f"Checkpoint   : {CHECKPOINT}")
print(f"Frames (local, cached): {FRAMES_DIR}")
print(f"Run (local)  : {RUN_LOCAL}")
print(f"Run (Drive)  : {RUN_DRIVE}  (written at the sync steps)")
```

```
Source video : /content/drive/MyDrive/DS340WCourseProject/FLAIR_code/FLAIR/example_video_blacktip.mp4
Checkpoint   : /content/drive/MyDrive/checkpoints/sam2_hiera_large.pt
Frames (local, cached): /content/flair_work/frames/example_video_blacktip
Run (local)  : /content/flair_work/runs/analysis_dev
Run (Drive)  : /content/drive/MyDrive/DS340WCourseProject/runs/analysis_dev  (written at the sync steps)
```

Load dependencies

```
import importlib.metadata as importlib_metadata
import importlib.util
import subprocess


def _run_shell(cmd):
    """Run a shell command quietly; on failure raise with the tail of its output."""
    result = subprocess.run(cmd, shell=True, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True
    if result.returncode != 0:
        tail = "\n".join(result.stdout.splitlines()[-25:])
        raise RuntimeError(f"Command failed ({result.returncode}): {cmd}\n{tail}")


def _torch_pinned():
    try:
        return importlib_metadata.version("torch").startswith("2.6.0")
    except importlib_metadata.PackageNotFoundError:
        return False


if _torch_pinned() and importlib.util.find_spec("sam2") and importlib.util.find_spec("transformers"):
    print("Dependencies already installed - skipping.")
else:
    print("Installing dependencies (a few minutes)...")
    _run_shell("apt-get update -qq && apt-get install -y -qq gcc g++")
    _run_shell("pip install -q numpy matplotlib pillow tqdm opencv-python")
    _run_shell("pip install -q torch==2.6.0 torchvision==0.21.0")
    _run_shell("pip install -q transformers==4.51.3 accelerate==1.7.0 omegaconf==2.3.0 iopath==0.1.10 "
               "huggingface-hub==0.31.4 regex==2024.11.6 portalocker==3.1.1")
    # SAM2 is built from source. If you see "sam2/_C.so: undefined symbol" later, SAM2's optional hole-f
    # post-processing is being skipped (results are otherwise fine). Adding --no-build-isolation may fix
    _run_shell("pip install -q git+https://github.com/conservation-technology-group/sam2.git")
    print("Done. If Colab asks you to restart the runtime, restart it and Run all again.")
```

```
Installing dependencies (a few minutes)...
Done. If Colab asks you to restart the runtime, restart it and Run all again.
```

```
import gc
import importlib.metadata as importlib_metadata
import random
import subprocess
import time
import urllib.request
```

```
import zipfile
from collections import defaultdict
from contextlib import contextmanager

import cv2
import matplotlib.pyplot as plt
import numpy as np
import torch
from matplotlib.patches import Rectangle
from PIL import Image
from tqdm import tqdm

if not torch.cuda.is_available():
    raise RuntimeError("No GPU detected. In Colab: Runtime > Change runtime type > pick a GPU, then Run
torch.autocast("cuda", dtype=torch.bfloat16).__enter__()   # SAM2 runs under bfloat16 autocast; enabled


# ---------------------------------------------------------------- generic helpers
timings = {}


@contextmanager
def timed(name):
    t0 = time.perf_counter()
    try:
        yield
    finally:
        timings[name] = round(time.perf_counter() - t0, 2)
        print(f"[time] {name}: {timings[name]} s")


def run_cmd(args):
    """Run a command (list form); on failure raise with the tail of its output."""
    result = subprocess.run(args, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True)
    if result.returncode != 0:
        tail = "\n".join(result.stdout.splitlines()[-25:])
        raise RuntimeError(f"Command failed ({result.returncode}): {' '.join(map(str, args))}\n{tail}")
    return result.stdout


def list_frames(frames_dir):
    """Frame image names, sorted numerically (frames must be named 00000.jpg, 00001.jpg, ...)."""
    return sorted(
        [p for p in os.listdir(frames_dir) if os.path.splitext(p)[-1].lower() in (".jpg", ".jpeg")],
        key=lambda p: int(os.path.splitext(p)[0]),
    )


def probe_fps(video_path):
    """Frames per second of a video via ffprobe (None if unavailable)."""
    try:
        out = run_cmd(["ffprobe", "-v", "error", "-select_streams", "v:0",
                       "-show_entries", "stream=r_frame_rate", "-of", "csv=p=0", video_path]).strip()
        num, _, den = out.partition("/")
        return float(num) / float(den or 1)
    except Exception:
        return None


# ---------------------------------------------------------------- mask / box helpers
def keep_masks(all_frames_masks, min_len, max_len):
    """Keep masks whose bbox width AND height are both between min_len and max_len."""
    keep = [
        i for i, m in enumerate(all_frames_masks)
        if min_len < m['bbox'][2] < max_len and min_len < m['bbox'][3] < max_len
    ]
    return [all_frames_masks[j] for j in keep]


def xywh_to_xyxy(bbox):
    return [int(bbox[0]), int(bbox[1]), int(bbox[0] + bbox[2]), int(bbox[1] + bbox[3])]
```

```
def bboxes_from_masks(masks, image_shape, buffer):
    """xyxy boxes around masks, padded by `buffer` pixels and clipped to the image."""
    bounding_boxes = []
    for m in masks:
        x_min, y_min, x_max, y_max = xywh_to_xyxy(m['bbox'])
        bounding_boxes.append([
            max(0, x_min - buffer),
            max(0, y_min - buffer),
            min(x_max + buffer, image_shape[1]),
            min(y_max + buffer, image_shape[0]),
        ])
    return bounding_boxes


def get_bounding_box(mask):
    """(x_min, y_min, x_max, y_max) of a boolean mask, or None if empty."""
    mask = np.squeeze(mask)
    y_idx, x_idx = np.where(mask)
    if not x_idx.size or not y_idx.size:
        return None
    return np.min(x_idx), np.min(y_idx), np.max(x_idx), np.max(y_idx)


def calculate_iou(b1, b2):
    xA, yA, xB, yB = max(b1[0], b2[0]), max(b1[1], b2[1]), min(b1[2], b2[2]), min(b1[3], b2[3])
    inter = max(0, xB - xA) * max(0, yB - yA)
    union = (b1[2] - b1[0]) * (b1[3] - b1[1]) + (b2[2] - b2[0]) * (b2[3] - b2[1]) - inter
    return inter / union if union > 0 else 0


def mask_to_polygons(mask):
    mask = np.squeeze(mask.astype(np.uint8))
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    polygons = []
    for contour in contours:
        if len(contour) >= 3:
            polygons.append(contour.reshape(-1, 2).tolist())
    return polygons


def polygons_to_mask(polygons, image_shape):
    mask = np.zeros(image_shape[:2], dtype=np.uint8)
    for polygon in polygons:
        if len(polygon) >= 3:
            cv2.fillPoly(mask, [np.array(polygon, dtype=np.int32)], 1)
    return mask.astype(bool)


# ---------------------------------------------------------------- saving results to Drive
_drive_initialised = False


def _zip_dir(src_dir, dst_zip):
    """Zip a flat folder of .npy files into one archive (one Drive write instead of hundreds) and verif
    names = sorted(f for f in os.listdir(src_dir) if f.endswith(".npy"))
    with zipfile.ZipFile(dst_zip, "w", zipfile.ZIP_DEFLATED) as z:
        for f in names:
            z.write(os.path.join(src_dir, f), arcname=f)
    with zipfile.ZipFile(dst_zip) as z:
        if z.testzip() is not None or len(z.namelist()) != len(names):
            raise RuntimeError(f"Archive verification failed: {dst_zip}")
    return len(names)


def sync_run_to_drive(stage):
    """Copy this run's results to PROJECT_DIR/runs/<RUN_NAME>/ on Drive. Safe to call repeatedly.

    Only ever touches RUN_DRIVE. With OVERWRITE=True the old folder is replaced the first time this is
    called in a session (i.e. only once new results exist)."""
```

```
global _drive_initialised
if os.path.dirname(os.path.realpath(RUN_DRIVE)) != os.path.realpath(f"{PROJECT_DIR}/runs"):
    raise RuntimeError(f"Refusing to write outside {PROJECT_DIR}/runs: {RUN_DRIVE}")
if not _drive_initialised:
    if os.path.exists(RUN_DRIVE):
        if not OVERWRITE:
            raise FileExistsError(f"Run folder already exists: {RUN_DRIVE} (set OVERWRITE = True or
        shutil.rmtree(RUN_DRIVE)
    _drive_initialised = True
os.makedirs(RUN_DRIVE, exist_ok=True)

saved = {}
for src_dir, zip_name in [(SEGMENTS_DIR, "video_segments.zip"), (PRUNED_DIR, "video_segments_pruned.
                          (SNAPSHOT_DIR, "video_segments_all_tracks.zip")]:
    if os.path.isdir(src_dir) and any(f.endswith(".npy") for f in os.listdir(src_dir)):
        saved[zip_name] = _zip_dir(src_dir, f"{RUN_DRIVE}/{zip_name}")
for path in (OUTPUT_TXT, OUTPUT_VIDEO):
    if os.path.isfile(path):
        shutil.copy2(path, RUN_DRIVE)
        saved[os.path.basename(path)] = os.path.getsize(path)
if os.path.isdir(ANALYSIS_DIR) and os.listdir(ANALYSIS_DIR):
    shutil.copytree(ANALYSIS_DIR, f"{RUN_DRIVE}/analysis", dirs_exist_ok=True)
    saved["analysis/"] = sorted(os.listdir(ANALYSIS_DIR))

info = {
    "stage": stage,
    "config": CONFIG,
    "timings_s": timings,
    "regression": globals().get("regression_result"),
    "funnel": globals().get("funnel"),
    "environment": {
        "torch": torch.__version__,
        "gpu": torch.cuda.get_device_name(0),
        "transformers": importlib_metadata.version("transformers"),
    },
}
with open(f"{RUN_DRIVE}/run_info.json", "w") as f:
    json.dump(info, f, indent=2, default=str)
print(f"[sync:{stage}] saved to {RUN_DRIVE}: {saved}")
```

SAM2 setup

```
def download_checkpoint_if_missing(path):
    """Download the SAM2 Large checkpoint; written via a temp file so an interrupted download is never m
    if os.path.exists(path):
        return
    os.makedirs(os.path.dirname(path), exist_ok=True)
    print(f"Downloading SAM2 checkpoint to {path} ...")
    tmp_path = path + ".part"
    urllib.request.urlretrieve("https://dl.fbaipublicfiles.com/segment_anything_2/072824/sam2_hiera_larg
    os.replace(tmp_path, path)
    print("Download complete.")


from sam2.build_sam import build_sam2, build_sam2_video_predictor
from sam2.automatic_mask_generator import SAM2AutomaticMaskGenerator

download_checkpoint_if_missing(CHECKPOINT)
device = torch.device("cuda")

# Video predictor (tracking/propagation) and automatic mask generator (candidate masks)
predictor = build_sam2_video_predictor(MODEL_CFG, CHECKPOINT, device=device, vos_optimized=VOS_OPTIMIZED
_sam = build_sam2(MODEL_CFG, CHECKPOINT, device=device, apply_postprocessing=False)
mask_generator = SAM2AutomaticMaskGenerator(model=_sam, **MASK_GENERATOR_PARAMS)
print("SAM2 loaded on", device)
```

```
SAM2 loaded on cuda
```

Extract video frames

Frames are written to local scratch and reused if the source video and clip settings have not changed.

```
    def extract_frames():
        meta = {"source": SOURCE_VIDEO, "mtime": os.path.getmtime(SOURCE_VIDEO), "start": CLIP_START, "end":
        marker = Path(FRAMES_DIR) / ".extract_meta.json"
        if not FORCE_REEXTRACT and marker.exists() and list_frames(FRAMES_DIR):
            try:
                if json.loads(marker.read_text()) == meta:
                    print(f"Using cached frames in {FRAMES_DIR} ({len(list_frames(FRAMES_DIR))} frames)")
                    return
            except ValueError:
                pass
        shutil.rmtree(FRAMES_DIR, ignore_errors=True)
        os.makedirs(FRAMES_DIR)
        cmd = ["ffmpeg", "-hide_banner", "-loglevel", "error", "-y"]
        if CLIP_START:
            cmd += ["-ss", CLIP_START]
        if CLIP_END:
            cmd += ["-to", CLIP_END]
        cmd += ["-i", SOURCE_VIDEO, "-q:v", "2", "-start_number", "0", f"{FRAMES_DIR}/%05d.jpg"]
        run_cmd(cmd)
        n_frames = len(list_frames(FRAMES_DIR))
        if n_frames == 0:
            raise RuntimeError(f"ffmpeg produced no frames from {SOURCE_VIDEO} (check CLIP_START / CLIP_END)
        marker.write_text(json.dumps(meta))
        print(f"Extracted {n_frames} frames to {FRAMES_DIR}")


    with timed("extract_frames"):
        extract_frames()

    frame_names = list_frames(FRAMES_DIR)
    SOURCE_FPS = probe_fps(SOURCE_VIDEO)
    if SOURCE_FPS and abs(SOURCE_FPS - RUN_EVERY_N_FRAMES) > 1:
        print(f"Note: source video is {SOURCE_FPS:.2f} fps but RUN_EVERY_N_FRAMES={RUN_EVERY_N_FRAMES}; "
              f"'Second N' labels below are every {RUN_EVERY_N_FRAMES} frames, not true seconds.")

    # One frame every RUN_EVERY_N_FRAMES frames is used for candidate-mask generation
    images = [
        np.array(Image.open(os.path.join(FRAMES_DIR, frame_names[i])).convert("RGB"))
        for i in range(0, len(frame_names), RUN_EVERY_N_FRAMES)
    ]
    image_shape = images[0].shape
    print(f"{len(frame_names)} frames, {len(images)} sampled for mask generation, frame size {image_shape[1]
```

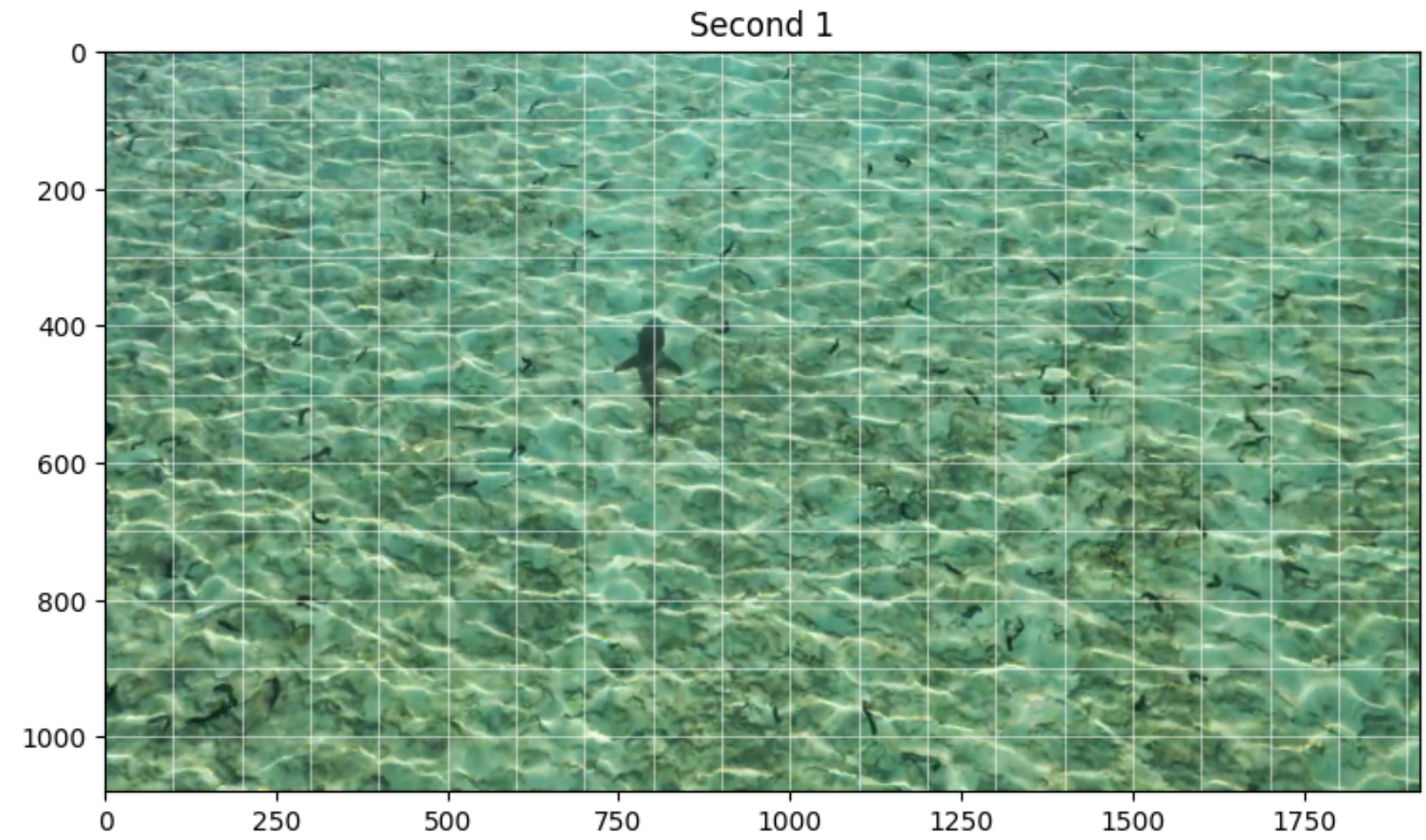
```
    Extracted 237 frames to /content/flair_work/frames/example_video_blacktip
    [time] extract_frames: 3.29 s
    237 frames, 8 sampled for mask generation, frame size 1920x1080
```

Estimate the minimum and maximum mask size from the gridded frame below (the object should fit within an n x n pixel box). If uncertain, set the minimum to 0 and the maximum to your estimated max. Edit `MIN_MASK_LENGTH` / `MAX_MASK_LENGTH` in the configuration cell and Run all again if they need changing.

```
# Show a sampled frame with a transparent grid every 100 pixels
second = min(1, len(images) - 1)
plt.figure(figsize=(9, 6))
plt.title(f"Second {second}")
plt.imshow(images[second])

height, width = images[0].shape[:2]
for x in range(0, width, 100):
    plt.axvline(x=x, color='white', linewidth=0.8, alpha=0.5)
for y in range(0, height, 100):
    plt.axhline(y=y, color='white', linewidth=0.8, alpha=0.5)

plt.show()
gc.collect()
```




```
16164
```

SAM2 automatic mask generation

Generate candidate masks on each sampled frame, keep those in the allowed size range, and turn them into padded boxes.

```
all_frames_bboxes = []
amg_counts = []   # (raw masks, masks after size filter) per sampled frame
with timed("mask_generation"):
    for img in tqdm(images):
        masks = mask_generator.generate(img)
        n_raw = len(masks)
        masks = keep_masks(masks, MIN_MASK_LENGTH, MAX_MASK_LENGTH)
        amg_counts.append((n_raw, len(masks)))
        all_frames_bboxes.append(bboxes_from_masks(masks, img.shape, BBOX_BUFFER))
        torch.cuda.empty_cache()
print("Candidate boxes per sampled frame:", [len(b) for b in all_frames_bboxes])
```

```
100%|██████████| 8/8 [00:46<00:00,  5.86s/it][time] mask_generation: 46.89 s
Candidate boxes per sampled frame: [1, 1, 1, 3, 5, 5, 2, 4]
```

CLIP filtering

Keep only candidate boxes that CLIP classifies as the target. Prompts are set in the configuration cell; some manual prompt tuning may be necessary.

```
from transformers import CLIPModel, CLIPProcessor

clip_model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
clip_processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32", use_fast=False)
```

```
/usr/local/lib/python3.13/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/se
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json:        4.19k/? [00:00<00:00, 484kB/s]
pytorch_model.bin: 100%                                                 605M/605M [00:03<00:00, 237MB/s]
preprocessor_config.json: 100%                                          316/316 [00:00<00:00, 42.0kB/s]
tokenizer_config.json: 100%                                             592/592 [00:00<00:00, 81.7kB/s]
vocab.json:         862k/? [00:00<00:00, 41.6MB/s]
merges.txt:         525k/? [00:00<00:00, 30.5MB/s]
model.safetensors: 100%                                                 605M/605M [00:01<00:00, 411MB/s]
special_tokens_map.json: 100%                                           389/389 [00:00<00:00, 49.4kB/s]
tokenizer.json:         2.22M/? [00:00<00:00, 52.3MB/s]
```

```
keep_boxes = []   # (sampled-frame index, box index within that frame, [x1, y1, x2, y2])
clip_records = []  # every candidate that CLIP scored (accepted or not), for the analysis section


def clip_scores(p, lg):
    """Target probability (best of CLIP_CORRECT prompts) and logit margin vs the strongest other prompt.
    others = [k for k in range(len(lg)) if k not in CLIP_CORRECT]
    return float(max(p[CLIP_CORRECT])), float(max(lg[CLIP_CORRECT]) - max(lg[others]))


with timed("clip_filtering"):
    for i, (img, bboxes) in enumerate(zip(images, all_frames_bboxes)):
        crops = [img[y1:y2 + 1, x1:x2 + 1] for x1, y1, x2, y2 in bboxes]
        if not crops:
            continue

        inputs = clip_processor(text=CLIP_PROMPTS, images=crops, return_tensors="pt", padding=True)
        outputs = clip_model(**inputs)
        probs = outputs["logits_per_image"].softmax(dim=1).detach().numpy()
        logits = outputs["logits_per_image"].detach().numpy()
        for j, (p, lg) in enumerate(zip(probs, logits)):
            p_target, margin = clip_scores(p, lg)
            clip_records.append({"frame_sampled": i, "box_idx": j, "bbox": [int(v) for v in bboxes[j]],
                                 "p_target": p_target, "margin": margin, "accepted": bool(p_target > CL

        for j, p in enumerate(probs):
            if (p[CLIP_CORRECT] > CLIP_CONFIDENCE).any():
                keep_boxes.append((i, j, bboxes[j]))

                # Show the accepted crop
                print("Second", i)
                plt.imshow(crops[j])
                plt.axis('off')
                plt.show()
```

```
                plt.close()
                gc.collect()

if not keep_boxes:
    raise RuntimeError(
        "CLIP kept 0 boxes. Try lowering CLIP_CONFIDENCE, adjusting CLIP_PROMPTS / CLIP_CORRECT, "
        "or widening MIN_MASK_LENGTH / MAX_MASK_LENGTH."
    )

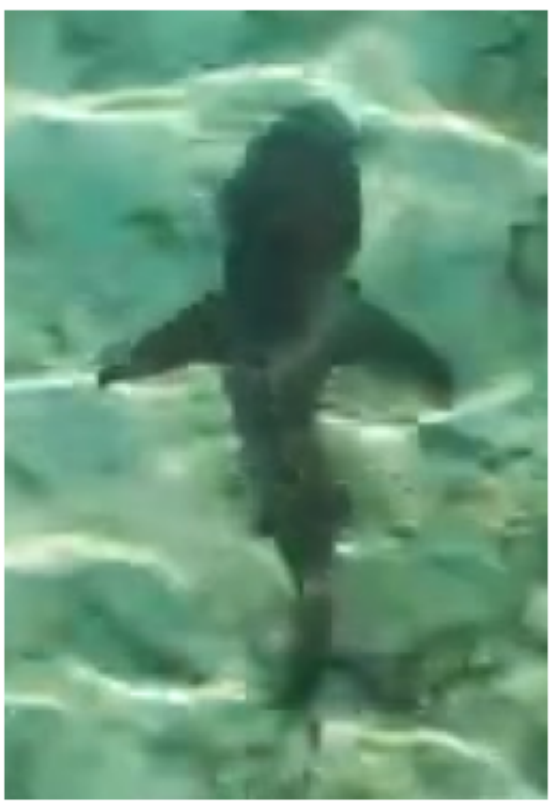
with open(OUTPUT_TXT, "w") as f:
    for i, j, bbox in keep_boxes:
        f.write(f"{i},{j},{','.join(map(str, bbox))}\n")
print(f"Saved {len(keep_boxes)} boxes to {OUTPUT_TXT}")

# Group by sampled frame
frame_keep_boxes = [[] for _ in range(len(images))]
for i, j, bbox in keep_boxes:
    frame_keep_boxes[i].append((i, j, bbox))
```

```
Second 0
```



```
Second 1
```

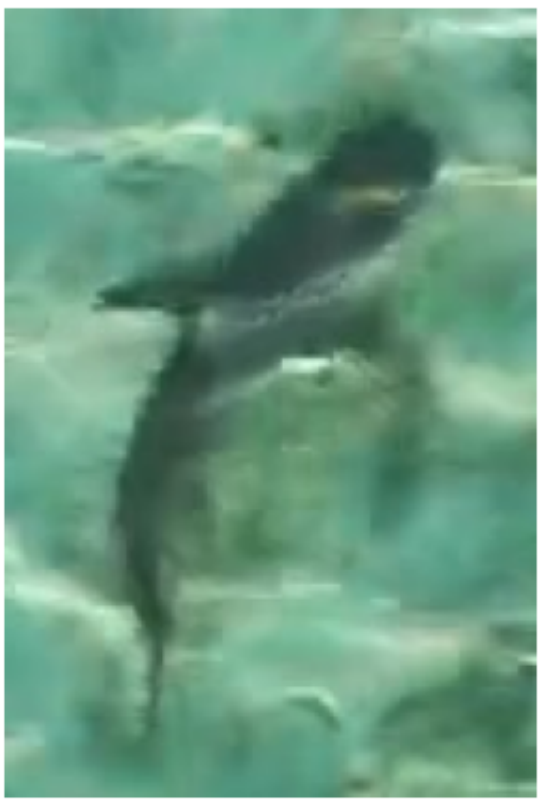


```
Second 2
```





Second 3



Second 3



Second 4





Second 4



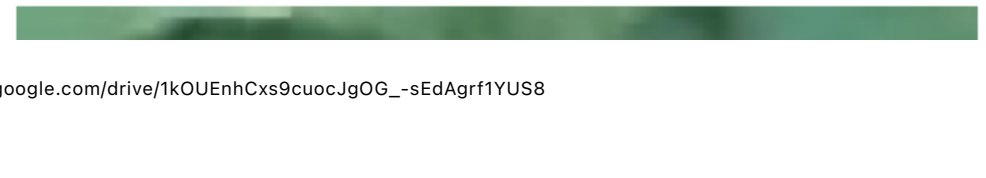
Second 4



Second 5



Second 5





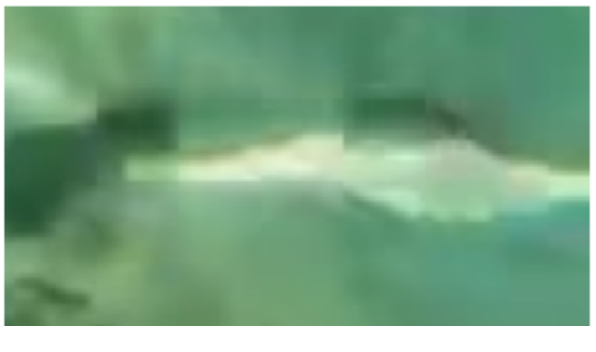
Second 6



Second 7



Second 7





```
[time] clip_filtering: 7.47 s
Saved 13 boxes to /content/flair_work/runs/analysis_dev/keep_boxes.txt
```

SAM2 video propagation + frame alignment

```
with torch.inference_mode(), torch.autocast("cuda", dtype=torch.bfloat16):
    inference_state = predictor.init_state(video_path=FRAMES_DIR, async_loading_frames=True)
print("Inference state ready")
```

```
Inference state ready
```

```
prop_log = []      # one entry per SAM2 propagate call: kind, track ids, frames processed
track_meta = {}    # track id -> sampled frame it was created at, its box, and whether it was confirmed


def _propagate(predictor, inference_state, output_dir, start, max_frame, image_shape, log_kind=None, log
    """Propagate prompted objects through the video and merge the resulting polygons into frame_<idx>.np
    all_results = defaultdict(dict)
    n_frames_passed = 0
    with torch.autocast("cuda", torch.bfloat16):
        with torch.inference_mode():
            iterator = predictor.propagate_in_video(
                inference_state,
                start_frame_idx=start,
                max_frame_num_to_track=max_frame,
            )
            for out_frame_idx, out_obj_ids, out_mask_logits in iterator:
                frame_segments = {
                    out_obj_id: mask_to_polygons((out_mask_logits[i] > 0.0).cpu().numpy())
                    for i, out_obj_id in enumerate(out_obj_ids)
                }
                all_results[out_frame_idx].update(frame_segments)
                n_frames_passed += 1
                del frame_segments, out_obj_ids, out_mask_logits

    for out_frame_idx, new_segments in all_results.items():
        file_path = os.path.join(output_dir, f'frame_{out_frame_idx}.npy')
        try:
            old_segments = np.load(file_path, allow_pickle=True).item()
            old_segments.update(new_segments)
            np.save(file_path, old_segments)
            del old_segments
        except FileNotFoundError:
            np.save(file_path, new_segments)
        del new_segments

    prop_log.append({"kind": log_kind, "ids": list(log_ids), "start": start, "frames": n_frames_passed})
    return n_frames_passed


def run_propagation(predictor, inference_state, frame_keep_boxes, image_shape, frame_gap,
                    window_length, output_dir, bbox_buffer, snapshot_dir=None):
    """FLAIR tracking: prompt SAM2 with CLIP-approved boxes, propagate over a window, de-duplicate and r
    os.makedirs(output_dir, exist_ok=True)
    start_time = time.time()
    curr_id = 0
```

```
        correct = []
        frames = []
        origin = []
        repropagate = []
        prop_log.clear()
        track_meta.clear()

        for a in range(len(frame_keep_boxes)):
            gc.collect()
            torch.cuda.empty_cache()

            print(f"\nSecond: {a}")

            if curr_id == 0:
                predictor.reset_state(inference_state)
                for b in range(len(frame_keep_boxes[a])):
                    curr_id += 1
                    i, j, bbox = frame_keep_boxes[a][b]
                    predictor.add_new_points_or_box(
                        inference_state=inference_state,
                        frame_idx=i * frame_gap,
                        obj_id=curr_id,
                        box=bbox,
                    )
                    correct.append(False)
                    frames.append(i)
                    origin.append(bbox)
                    repropagate.append(i + window_length)
            if curr_id != 0:
                _propagate(
                    predictor,
                    inference_state,
                    output_dir,
                    log_kind="new", log_ids=list(range(1, curr_id + 1)),
                    start=max(0, frame_gap * (a - window_length)),
                    max_frame=(frame_gap * window_length) + (frame_gap * a - max(0, frame_gap * (a - wir
                    image_shape=image_shape,
                )

        else:
            prev_curr_id = curr_id
            predictor.reset_state(inference_state)

            for b in range(len(frame_keep_boxes[a])):
                i, j, bbox = frame_keep_boxes[a][b]
                filename = os.path.join(output_dir, f'frame_{i * frame_gap}.npy')
                mask_add = True

                if os.path.exists(filename):
                    frame_segments_poly = np.load(filename, allow_pickle=True).item()
                    frame_segments = {k: polygons_to_mask(v, image_shape) for k, v in frame_segments_pol
                    for key in sorted(frame_segments.keys(), key=int):
                        bounding_box = get_bounding_box(frame_segments[key])
                        if bounding_box is not None:
                            bounding_box = [
                                max(0, bounding_box[0] - bbox_buffer),
                                max(0, bounding_box[1] - bbox_buffer),
                                min(bounding_box[2] + bbox_buffer, image_shape[1]),
                                min(bounding_box[3] + bbox_buffer, image_shape[0]),
                            ]
                            iou = calculate_iou(bbox, bounding_box)
                            if iou > 0.7:
                                print("Duplicate mask!")
                                mask_add = False
                                if i != frames[int(key) - 1]:
                                    correct[int(key) - 1] = True
                                break
                    del frame_segments

                if mask_add:
                    curr_id += 1
```

```
                    predictor.add_new_points_or_box(
                        inference_state=inference_state,
                        frame_idx=i * frame_gap,
                        obj_id=curr_id,
                        box=bbox,
                    )
                    correct.append(False)
                    frames.append(i)
                    origin.append(bbox)
                    repropagate.append(i + window_length)

            if prev_curr_id != curr_id:
                _propagate(
                    predictor,
                    inference_state,
                    output_dir,
                    log_kind="new", log_ids=list(range(prev_curr_id + 1, curr_id + 1)),
                    start=max(0, frame_gap * (a - window_length)),
                    max_frame=(frame_gap * window_length) + (frame_gap * a - max(0, frame_gap * (a - wir
                    image_shape=image_shape,
                )

        # Re-propagate tracks that were confirmed by a later detection
        indices = [i for i, x in enumerate(repropagate) if x == a]
        should_reprop = False
        reprop_ids = []
        predictor.reset_state(inference_state)

        for index in indices:
            if correct[index]:
                filename = os.path.join(output_dir, f'frame_{(frame_gap * a) - 1}.npy')
                frame_segments_old_poly = np.load(filename, allow_pickle=True).item()
                frame_segments_old = {k: polygons_to_mask(v, image_shape) for k, v in frame_segments_old
                bounding_box = get_bounding_box(frame_segments_old.get(index + 1))
                del frame_segments_old

                if bounding_box is not None:
                    should_reprop = True
                    reprop_ids.append(index + 1)
                    print("Re-propagating index", index, "at second", a)
                    bounding_box = [
                        max(0, bounding_box[0] - bbox_buffer),
                        max(0, bounding_box[1] - bbox_buffer),
                        min(bounding_box[2] + bbox_buffer, image_shape[1]),
                        min(bounding_box[3] + bbox_buffer, image_shape[0]),
                    ]

                    predictor.add_new_points_or_box(
                        inference_state=inference_state,
                        frame_idx=(frame_gap * a),
                        obj_id=index + 1,
                        box=bounding_box,
                    )
                    repropagate[index] += window_length

            if should_reprop:
                _propagate(
                    predictor,
                    inference_state,
                    output_dir,
                    log_kind="reprop", log_ids=reprop_ids,
                    start=frame_gap * a,
                    max_frame=frame_gap * window_length,
                    image_shape=image_shape,
                )

        # [analysis] record every track and keep a copy of all of them before the final filter deletes the u
        for idx in range(curr_id):
            track_meta[idx + 1] = {"created_sampled_frame": frames[idx], "origin_box": [int(v) for v in orig
                                   "confirmed": bool(correct[idx])}
        if snapshot_dir:
```

```
            shutil.rmtree(snapshot_dir, ignore_errors=True)
            shutil.copytree(output_dir, snapshot_dir)

        # Keep only tracks that were confirmed by a second detection
        correct_indices = [i + 1 for i, is_correct in enumerate(correct) if is_correct]
        npy_files = [f for f in os.listdir(output_dir) if f.endswith(".npy")]
        for fname in tqdm(npy_files, desc="Filtering files"):
            fpath = os.path.join(output_dir, fname)
            data = np.load(fpath, allow_pickle=True).item()
            filtered = {k: v for k, v in data.items() if int(k) in correct_indices}
            np.save(fpath, filtered)

        print(f"Window {window_length} took {time.time() - start_time:.2f} seconds")
```

```
# Propagate masks through the video (a few minutes on a GPU)
with timed("propagation"):
    run_propagation(
        predictor=predictor,
        inference_state=inference_state,
        frame_keep_boxes=frame_keep_boxes,
        image_shape=image_shape,
        frame_gap=RUN_EVERY_N_FRAMES,
        window_length=WINDOW_LENGTH,
        output_dir=SEGMENTS_DIR,
        bbox_buffer=BBOX_BUFFER,
        snapshot_dir=SNAPSHOT_DIR,
    )

if not any(f.endswith(".npy") for f in os.listdir(SEGMENTS_DIR)):
    raise RuntimeError("Propagation produced no masks. Check the CLIP/size settings above.")

sync_run_to_drive("after_propagation")   # save the expensive step to Drive straight away
```

```
/usr/local/lib/python3.13/dist-packages/sam2/sam2_video_predictor.py:786: UserWarning: /usr/local/lib/pyt

Skipping the post-processing step due to the error above. You can still use SAM 2 and it's OK to ignore t
  pred_masks_gpu = fill_holes_in_mask_scores(

Second: 0
propagate in video: 100%|██████████| 91/91 [00:14<00:00,  6.28it/s]

Second: 1
propagate in video: 100%|██████████| 121/121 [00:19<00:00,  6.29it/s]

Second: 2
Duplicate mask!

Second: 3
Duplicate mask!
propagate in video: 100%|██████████| 181/181 [00:29<00:00,  6.23it/s]

Second: 4
Duplicate mask!
propagate in video: 100%|██████████| 181/181 [00:33<00:00,  5.42it/s]
Re-propagating index 1 at second 4
propagate in video: 100%|██████████| 91/91 [00:14<00:00,  6.24it/s]

Second: 5
propagate in video: 100%|██████████| 177/177 [00:32<00:00,  5.40it/s]

Second: 6
Duplicate mask!

Second: 7
Duplicate mask!
propagate in video: 100%|██████████| 117/117 [00:18<00:00,  6.22it/s]
Re-propagating index 1 at second 7
propagate in video: 100%|██████████| 27/27 [00:04<00:00,  6.37it/s]
Filtering files: 100%|██████████| 237/237 [00:00<00:00, 2301.33it/s]
Window 3 took 171.13 seconds
[time] propagation: 171.13 s
[sync:after_propagation] saved to /content/drive/MyDrive/DS340WCourseProject/runs/analysis_dev: {'video_s
```

Review tracks

Each track (mask ID) is shown with example frames. Note which IDs are true positives and set `KEEP_KEYS` in the configuration cell.

```
def display_mask_grid(n_per_mask, resize_factor, mask_dir, frames_dir, frame_names, image_shape):
    """Show n_per_mask example frames (with the mask's bounding box) for every track."""
    all_samples = defaultdict(list)
    for fname in tqdm(sorted(os.listdir(mask_dir)), desc="Indexing masks"):
        if not fname.endswith(".npy"):
            continue
        frame_index = int(fname.split("_")[1].split(".")[0])
        data = np.load(os.path.join(mask_dir, fname), allow_pickle=True).item()
        for mask_num, mask_polygons in data.items():
            mask = polygons_to_mask(mask_polygons, image_shape)
            bbox = get_bounding_box(mask)
            if bbox is not None:
                all_samples[mask_num].append((frame_index, mask, bbox))
        del data
        gc.collect()

    if not all_samples:
        print("No tracks to display.")
    for mask_num, samples in all_samples.items():
        if len(samples) < n_per_mask:
            continue
        selected = random.sample(samples, n_per_mask)
        frame_indices = [f for f, _, _ in samples]

        fig, axes = plt.subplots(3, 3, figsize=(12, 9))
        axes = axes.flatten()
        for ax, (frame_index, mask, bbox) in zip(axes, selected):
            image = Image.open(os.path.join(frames_dir, frame_names[frame_index])).convert("RGB")
            if resize_factor != 1.0:
                image = image.resize((int(image.width * resize_factor), int(image.height * resize_factor
            img_np = np.array(image)
            bbox_scaled = tuple(int(c * resize_factor) for c in bbox)

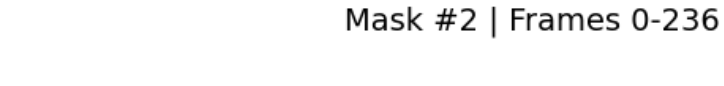
            pad = 5
            x_min = max(bbox_scaled[0] - pad, 0)
            y_min = max(bbox_scaled[1] - pad, 0)
            x_max = min(bbox_scaled[2] + pad, img_np.shape[1])
            y_max = min(bbox_scaled[3] + pad, img_np.shape[0])

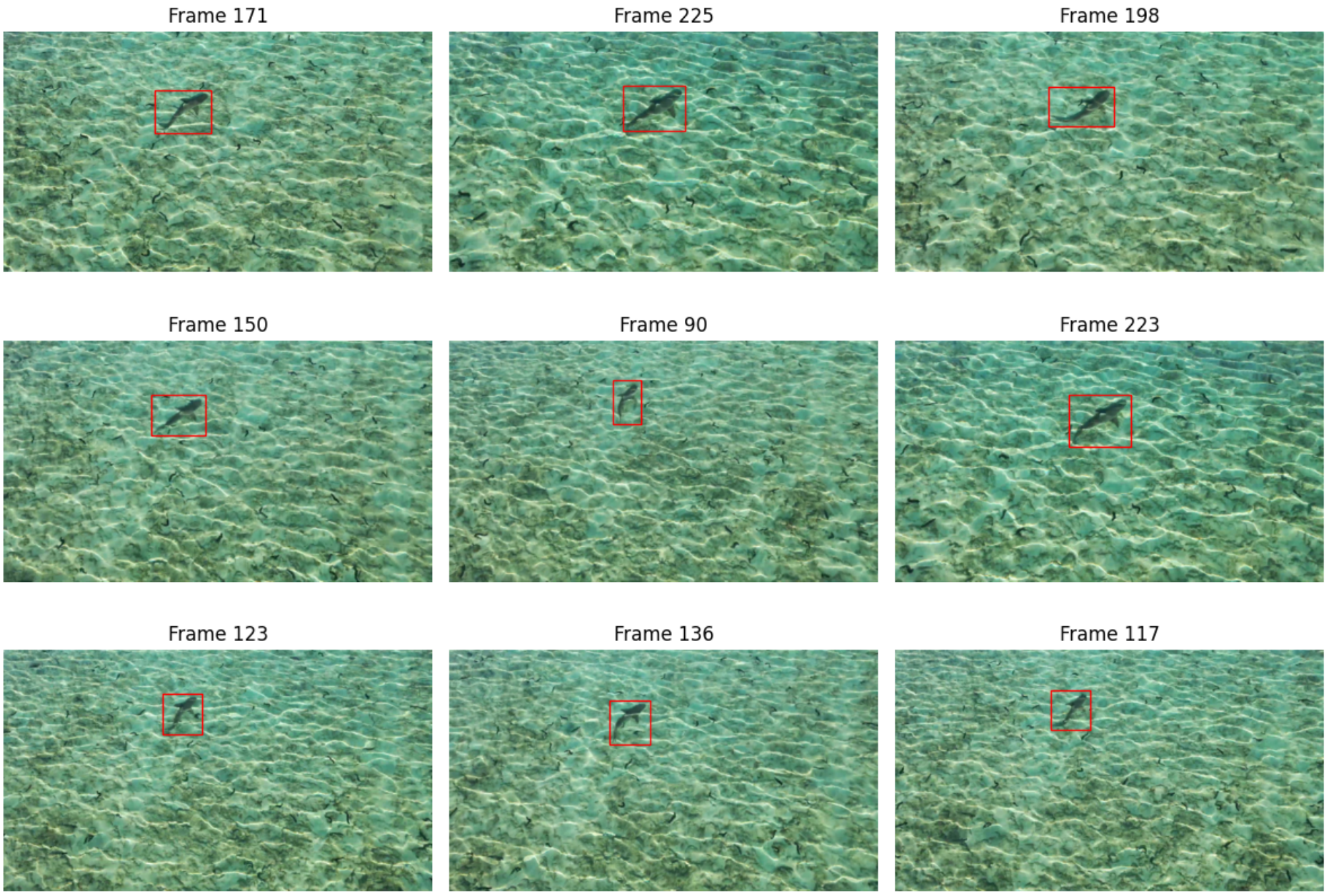
            ax.imshow(img_np)
            ax.add_patch(Rectangle((x_min, y_min), x_max - x_min, y_max - y_min,
                                   linewidth=1, edgecolor='red', facecolor='none'))
            ax.set_title(f"Frame {frame_index}")
            ax.axis("off")

        fig.suptitle(f"Mask #{mask_num} | Frames {min(frame_indices)}-{max(frame_indices)}", fontsize=14
        plt.tight_layout()
        plt.show()
        gc.collect()


display_mask_grid(
    n_per_mask=N_PER_MASK,
    resize_factor=RESIZE_FACTOR,
    mask_dir=SEGMENTS_DIR,
    frames_dir=FRAMES_DIR,
    frame_names=frame_names,
    image_shape=image_shape,
)
```

```
Indexing masks: 100%|██████████| 237/237 [01:29<00:00,  2.66it/s]
```





Mask pruning

Keep only the tracks listed in `KEEP_KEYS` (configuration cell).

```
all_masks = {}
for fname in sorted(os.listdir(SEGMENTS_DIR)):
    if fname.endswith(".npy"):
        all_masks[fname] = np.load(os.path.join(SEGMENTS_DIR, fname), allow_pickle=True).item()

available_ids = sorted({k for data in all_masks.values() for k in data})
keep_set = set(KEEP_KEYS)
if not (keep_set & set(available_ids)):
    raise ValueError(
        f"KEEP_KEYS {sorted(keep_set)} matches none of the tracks found: {available_ids}.\n"
        f"Look at the review grid above and set KEEP_KEYS in the configuration cell."
    )
if keep_set - set(available_ids):
    print(f"Warning: KEEP_KEYS {sorted(keep_set - set(available_ids))} not found (available: {available_

os.makedirs(PRUNED_DIR, exist_ok=True)
for fname, data in tqdm(all_masks.items(), desc="Pruning"):
    np.save(os.path.join(PRUNED_DIR, fname), {k: v for k, v in data.items() if k in keep_set})
print(f"Kept tracks {sorted(keep_set & set(available_ids))} of {available_ids}")
del all_masks
gc.collect()
```

```
Pruning: 100%|██████████| 237/237 [00:00<00:00, 7511.22it/s]Kept tracks [2] of [2]

20319
```

Visualization

Overlay the pruned masks on every frame and encode a video.

```
os.makedirs(OVERLAY_DIR, exist_ok=True)

COLORS = [  # BGR
    (0, 0, 255),     # red
    (0, 255, 0),     # green
    (255, 0, 255),   # magenta
    (255, 0, 0),     # blue
    (0, 255, 255),   # yellow
    (255, 165, 0),   # orange
    (128, 0, 128),   # purple
]
mask_colors = {k: COLORS[n % len(COLORS)] for n, k in enumerate(sorted(KEEP_KEYS))}


def save_frame_with_masks_cv2(frame_path, masks, output_path):
    """Overlay each mask in its own colour on the frame and save with OpenCV."""
    image = cv2.imread(frame_path)
    for key, mask in masks.items():
        overlay = np.zeros_like(image, dtype=np.uint8)
        overlay[mask] = mask_colors.get(key, COLORS[0])
        image = cv2.addWeighted(image, 1.0, overlay, 0.4, 0)
    cv2.imwrite(output_path, image)


for frame_idx in tqdm(range(len(frame_names)), desc="Overlaying masks"):
    mask_path = os.path.join(PRUNED_DIR, f'frame_{frame_idx}.npy')
    masks = {}
    if os.path.exists(mask_path):
        polygons = np.load(mask_path, allow_pickle=True).item()
        masks = {k: polygons_to_mask(v, image_shape) for k, v in polygons.items()}

    save_frame_with_masks_cv2(
        os.path.join(FRAMES_DIR, frame_names[frame_idx]),
        masks,
        os.path.join(OVERLAY_DIR, f'{frame_idx:05d}.jpg'),
    )
    if frame_idx % 100 == 0:
        gc.collect()
```

```
Overlaying masks: 100%|██████████| 237/237 [00:08<00:00, 29.03it/s]
```

```
output_fps = SOURCE_FPS or 30
run_cmd([
    "ffmpeg", "-hide_banner", "-loglevel", "error", "-y",
    "-framerate", f"{output_fps:g}", "-start_number", "0", "-i", f"{OVERLAY_DIR}/%05d.jpg",
    "-vf", "scale=ceil(iw/2)*2:ceil(ih/2)*2", "-c:v", "libx264", "-pix_fmt", "yuv420p", OUTPUT_VIDEO,
])
print(f"Saved {OUTPUT_VIDEO} ({os.path.getsize(OUTPUT_VIDEO) / 1e6:.1f} MB)")
```

```
Saved /content/flair_work/runs/analysis_dev/tracked_video.mp4 (11.4 MB)
```

```
from base64 import b64encode
from IPython.display import HTML

# Inline preview (skipped for large files - the video is saved to Drive at the final sync step anyway)
if os.path.getsize(OUTPUT_VIDEO) > 25e6:
    print("Video is large; skipping the inline preview.")
    preview = None
else:
    with open(OUTPUT_VIDEO, "rb") as f:
        data_url = "data:video/mp4; base64," + b64encode(f.read()).decode()
    preview = HTML(f"""<video width="1024" height="768" controls><source src="{data_url}" type="video/mp
preview
```

Regression check (optional)

Compares this run's pruned masks with the pruned masks from the original notebook (`BASELINE_PRUNED_DIR`, read-only). Tracks are compared as the union of all kept masks per frame, so track IDs do not need to match. Skipped if no baseline exists.

```
def _union_mask(npy_path):
    data = np.load(npy_path, allow_pickle=True).item()
    union = np.zeros(image_shape[:2], dtype=bool)
    for polygons in data.values():
        union |= polygons_to_mask(polygons, image_shape)
    return union


regression_result = None
if not os.path.isdir(BASELINE_PRUNED_DIR):
    print(f"No baseline found at {BASELINE_PRUNED_DIR} - skipping regression check.")
else:
    base_files = {f for f in os.listdir(BASELINE_PRUNED_DIR) if f.endswith(".npy")}
    new_files = {f for f in os.listdir(PRUNED_DIR) if f.endswith(".npy")}
    dice_scores, both_empty = [], 0
    for fname in sorted(base_files & new_files):
        a = _union_mask(os.path.join(BASELINE_PRUNED_DIR, fname))
        b = _union_mask(os.path.join(PRUNED_DIR, fname))
        if not a.any() and not b.any():
            both_empty += 1
            continue
        dice_scores.append(2.0 * np.logical_and(a, b).sum() / (a.sum() + b.sum()))

    if not dice_scores:
        print("No overlapping non-empty frames to compare.")
    else:
        mean_dice, min_dice = float(np.mean(dice_scores)), float(np.min(dice_scores))
        passed = mean_dice >= REGRESSION_MIN_DICE
        regression_result = {
            "frames_compared": len(dice_scores), "frames_both_empty": both_empty,
            "frames_only_in_baseline": len(base_files - new_files), "frames_only_in_new": len(new_files
            "mean_dice": round(mean_dice, 4), "min_dice": round(min_dice, 4),
            "threshold": REGRESSION_MIN_DICE, "passed": passed,
        }
        print(("PASS" if passed else "FAIL"), regression_result)
```

```
PASS {'frames_compared': 237, 'frames_both_empty': 0, 'frames_only_in_baseline': 0, 'frames_only_in_new':
```

Analysis (read-only)

Everything below only **reads** what the pipeline produced; it does not change any mask. It answers three questions about this run, and saves the numbers to `PROJECT_DIR/runs/<RUN_NAME>/analysis/`:

1. **Funnel / compute** - how many masks, CLIP candidates and tracks were created and kept, and how many SAM2 frame-passes were spent (and on which tracks).
2. **Track table** - for *every* track FLAIR created (including the ones it deleted at the end) simple temporal and geometric features, plus CLIP re-scored on a few frames.
3. **CLIP candidates** - how the CLIP score separates candidate boxes that sit on the track you kept (`KEEP_KEYS`) from the ones that do not, comparing FLAIR's softmax probability with a logit margin.

How to read the results. This is one clip. The numbers are descriptive and show what the measurements look like; they are not a validated classifier or a tuned threshold. The labels in (3) come from your own `KEEP_KEYS` choice, so they are only as good as that review.

1. Funnel and compute

```
import csv

os.makedirs(ANALYSIS_DIR, exist_ok=True)

retained_ids = sorted(k for k, m in track_meta.items() if m["confirmed"])
discarded_ids = sorted(k for k, m in track_meta.items() if not m["confirmed"])
n_video_frames = len(frame_names)
```

```
def _new_track_cost(predicate):
    """SAM2 frame-passes spent in propagate calls that were launched for newly added tracks and satisfy
    return sum(c["frames"] for c in prop_log if c["kind"] == "new" and c["ids"] and predicate(c["ids"]))


passes_total = sum(c["frames"] for c in prop_log)
funnel = {
    "sampled_frames": len(images),
    "video_frames": n_video_frames,
    "amg_masks_raw": sum(r for r, _ in amg_counts),
    "amg_masks_after_size_filter": sum(k for _, k in amg_counts),
    "clip_candidates_scored": len(clip_records),
    "clip_candidates_accepted": sum(1 for r in clip_records if r["accepted"]),
    "tracks_created": len(track_meta),
    "tracks_confirmed": len(retained_ids),
    "tracks_discarded_by_flair": len(discarded_ids),
    "tracks_kept_by_KEEP_KEYS": len(set(KEEP_KEYS) & set(retained_ids)),
    "propagate_calls": len(prop_log),
    "propagation_frame_passes": passes_total,
    "frame_passes_per_video_frame": round(passes_total / max(1, n_video_frames), 2),
    "frame_passes_new_calls_only_confirmed_tracks": _new_track_cost(lambda ids: all(i in retained_ids fo
    "frame_passes_new_calls_only_discarded_tracks": _new_track_cost(lambda ids: all(i in discarded_ids f
    "frame_passes_new_calls_mixed": _new_track_cost(
        lambda ids: any(i in retained_ids for i in ids) and any(i in discarded_ids for i in ids)),
    "frame_passes_repropagation": sum(c["frames"] for c in prop_log if c["kind"] == "reprop"),
    "stage_seconds": {k: timings[k] for k in ("mask_generation", "clip_filtering", "propagation") if k i
}
if funnel["clip_candidates_accepted"] != len(keep_boxes):
    print("WARNING: accepted-candidate count does not match keep_boxes - check the CLIP cell.")


def _row(label, value, note=""):
    print(f"  {label:<48}{value!s:>8}   {note}")


print("Pipeline funnel")
_row("sampled frames (mask generation)", funnel["sampled_frames"], f"of {n_video_frames} video frames")
_row("SAM2 masks generated (raw)", funnel["amg_masks_raw"])
_row("  after size filter", funnel["amg_masks_after_size_filter"], f"{MIN_MASK_LENGTH}-{MAX_MASK_LENGTH}
_row("  CLIP candidates scored", funnel["clip_candidates_scored"])
_row("  CLIP candidates accepted", funnel["clip_candidates_accepted"], f"confidence > {CLIP_CONFIDENCE}"
_row("tracks created", funnel["tracks_created"], "new objects prompted into SAM2 (duplicates excluded)")
_row("  confirmed by a later detection (retained)", funnel["tracks_confirmed"])
_row("  discarded by FLAIR's final filter", funnel["tracks_discarded_by_flair"])
_row("  kept by KEEP_KEYS", funnel["tracks_kept_by_KEEP_KEYS"])
print("\nSAM2 propagation work")
_row("propagate calls", funnel["propagate_calls"])
_row("frame-passes (frames processed, summed)", funnel["propagation_frame_passes"],
     f"= {funnel['frame_passes_per_video_frame']}x the {n_video_frames} video frames")
_row("  in calls for new tracks, all later confirmed", funnel["frame_passes_new_calls_only_confirmed_tra
_row("  in calls for new tracks, all later discarded", funnel["frame_passes_new_calls_only_discarded_tra
     "compute spent only on tracks FLAIR deleted")
_row("  in calls mixing both kinds", funnel["frame_passes_new_calls_mixed"], "cost cannot be split")
_row("  re-propagation of confirmed tracks", funnel["frame_passes_repropagation"])
print("\nStage time (s):", funnel["stage_seconds"])
print("\nNote: one clip. Frame-passes depend on how many detections occur and when; they are FLAIR's own

with open(f"{ANALYSIS_DIR}/funnel.json", "w") as f:
    json.dump(funnel, f, indent=2)
```

```
Pipeline funnel
  sampled frames (mask generation)                        8   of 237 video frames
  SAM2 masks generated (raw)                            435
    after size filter                                    22   75-300 px
    CLIP candidates scored                               22
    CLIP candidates accepted                             13   confidence > 0.95
  tracks created                                          8   new objects prompted into SAM2 (duplicates e
    confirmed by a later detection (retained)             1
    discarded by FLAIR's final filter                     7
    kept by KEEP_KEYS                                     1
```

```
SAM2 propagation work
  propagate calls                                      8
  frame-passes (frames processed, summed)            986   = 4.16x the 237 video frames
    in calls for new tracks, all later confirmed     121
    in calls for new tracks, all later discarded     747   compute spent only on tracks FLAIR deleted
    in calls mixing both kinds                         0   cost cannot be split
    re-propagation of confirmed tracks               118

Stage time (s): {'mask_generation': 46.89, 'clip_filtering': 7.47, 'propagation': 171.13}

Note: one clip. Frame-passes depend on how many detections occur and when; they are FLAIR's own design, r
```

2. Track table (every track FLAIR created, including discarded ones)

`frames_nonempty` mostly reflects FLAIR's propagation windows (a track that is never confirmed is only propagated once), so it largely restates the confirmation decision and is **not independent evidence** that something is a shark. The geometry columns (`area_cv`, `aspect_cv`, `disp_per_body`) and the CLIP re-scores (`clip_p_target`, `clip_margin`, medians over `TRACK_CLIP_SAMPLES` evenly spaced frames) are the more informative columns.

```
def _track_series(snapshot_dir):
    """Per track: rows (frame, area, x1, y1, x2, y2, cx, cy) for frames where its mask is non-empty."""
    series = defaultdict(list)
    for fname in tqdm(sorted(os.listdir(snapshot_dir)), desc="Reading all-track snapshot"):
        if not fname.endswith(".npy"):
            continue
        frame_idx = int(fname.split("_")[1].split(".")[0])
        data = np.load(os.path.join(snapshot_dir, fname), allow_pickle=True).item()
        for tid, polys in data.items():
            mask = polygons_to_mask(polys, image_shape)
            bb = get_bounding_box(mask)
            if bb is None:
                continue
            ys, xs = np.nonzero(mask)
            series[int(tid)].append((frame_idx, int(mask.sum()), int(bb[0]), int(bb[1]), int(bb[2]), int
                                     float(xs.mean()), float(ys.mean())))
        del data
    for tid in series:
        series[tid].sort()
    return series


def _cv(values):
    v = np.asarray(values, dtype=float)
    return float(v.std() / v.mean()) if v.size and v.mean() > 0 else float("nan")
```

```
def _rescore_track(rows):
    """Median CLIP target probability and logit margin over evenly spaced frames of one track."""
    if not rows or TRACK_CLIP_SAMPLES <= 0:
        return float("nan"), float("nan")
    n_pick = min(TRACK_CLIP_SAMPLES, len(rows))
    picks = [rows[k] for k in sorted(set(np.linspace(0, len(rows) - 1, n_pick).round().astype(int)))]
    crops = []
    for frame_idx, _, x1, y1, x2, y2, _, _ in picks:
        img = np.array(Image.open(os.path.join(FRAMES_DIR, frame_names[frame_idx])).convert("RGB"))
        x1p, y1p = max(0, x1 - BBOX_BUFFER), max(0, y1 - BBOX_BUFFER)
        x2p, y2p = min(x2 + BBOX_BUFFER, image_shape[1]), min(y2 + BBOX_BUFFER, image_shape[0])
        crops.append(img[y1p:y2p + 1, x1p:x2p + 1])
    inputs = clip_processor(text=CLIP_PROMPTS, images=crops, return_tensors="pt", padding=True)
    with torch.inference_mode():
        outputs = clip_model(**inputs)
    probs = outputs["logits_per_image"].softmax(dim=1).detach().numpy()
    logits = outputs["logits_per_image"].detach().numpy()
    scores = [clip_scores(p, lg) for p, lg in zip(probs, logits)]
    return float(np.median([s[0] for s in scores])), float(np.median([s[1] for s in scores]))
```

```
series = _track_series(SNAPSHOT_DIR)
track_rows = []
for tid in sorted(track_meta):
    meta, rows = track_meta[tid], series.get(tid, [])
    areas = [r[1] for r in rows]
    widths = [r[4] - r[2] + 1 for r in rows]
    heights = [r[5] - r[3] + 1 for r in rows]
    steps = [np.hypot(b[6] - a[6], b[7] - a[7]) for a, b in zip(rows, rows[1:]) if b[0] - a[0] == 1]
    p_med, m_med = _rescore_track(rows)
    status = ("kept (KEEP_KEYS)" if tid in KEEP_KEYS else
              "confirmed, not kept" if meta["confirmed"] else "discarded by FLAIR")
    track_rows.append({
        "track_id": tid, "status": status, "created_at_sampled_frame": meta["created_sampled_frame"],
        "frames_nonempty": len(rows), "frac_of_clip": round(len(rows) / max(1, n_video_frames), 3),
        "first_frame": rows[0][0] if rows else "", "last_frame": rows[-1][0] if rows else "",
        "area_mean": round(float(np.mean(areas)), 1) if rows else float("nan"),
        "area_cv": round(_cv(areas), 3),
        "aspect_cv": round(_cv([w / h for w, h in zip(widths, heights)]), 3),
        "disp_per_body": round(float(np.mean(steps) / np.sqrt(np.mean(areas))), 4) if steps else float('
        "clip_p_target": round(p_med, 3), "clip_margin": round(m_med, 2),
    })

if track_rows:
    cols = list(track_rows[0])
    print("  ".join(f"{c:>{max(len(c), 8)}}" for c in cols))
    for r in track_rows:
        print("  ".join(f"{str(r[c]):>{max(len(c), 8)}}" for c in cols))
    with open(f"{ANALYSIS_DIR}/track_features.csv", "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=cols)
        writer.writeheader()
        writer.writerows(track_rows)
    if len(track_rows) == 1:
        print("\nOnly one track exists in this run, so there is nothing to compare it against.")
    elif not discarded_ids:
        print("\nNo track was discarded in this run, so there is no false track to compare against.")
else:
    print("No tracks to tabulate.")
```

```
Reading all-track snapshot: 100%|██████████| 237/237 [00:12<00:00, 19.32it/s]
track_id    status  created_at_sampled_frame  frames_nonempty  frac_of_clip  first_frame  last_frame  are
       1  discarded by FLAIR                         0               91         0.384            0
       2  kept (KEEP_KEYS)                           1              237           1.0            0
       3  discarded by FLAIR                         3               21         0.089            0
       4  discarded by FLAIR                         4               38          0.16          103
       5  discarded by FLAIR                         4               23         0.097           30
       6  discarded by FLAIR                         5              177         0.747           60
       7  discarded by FLAIR                         5                8         0.034          149
       8  discarded by FLAIR                         7               32         0.135          205
```

3. CLIP candidates: on the kept track vs. not

Every candidate box that passed the size filter was scored by CLIP. A candidate is labelled **on kept track** if its box overlaps (IoU >= `CANDIDATE_LABEL_IOU`) the padded box of a `KEEP_KEYS` track at that frame, otherwise **not on kept track**. Two scores are compared: FLAIR's softmax probability of the target prompt(s), and a logit margin (best target prompt minus the strongest other prompt). With so few candidates this describes the data; it does not justify a threshold.

```
def _kept_boxes_at(frame_idx):
    """Padded boxes (same padding as FLAIR) of the KEEP_KEYS tracks at this video frame, from the all-t
    path = os.path.join(SNAPSHOT_DIR, f"frame_{frame_idx}.npy")
    if not os.path.exists(path):
        return []
    data = np.load(path, allow_pickle=True).item()
    boxes = []
    for tid, polys in data.items():
        if int(tid) not in KEEP_KEYS:
            continue
        bb = get_bounding_box(polygons_to_mask(polys, image_shape))
        if bb is not None:
```

```
            boxes.append([max(0, bb[0] - BBOX_BUFFER), max(0, bb[1] - BBOX_BUFFER),
                          min(bb[2] + BBOX_BUFFER, image_shape[1]), min(bb[3] + BBOX_BUFFER, image_shape
    return boxes


cand_rows = []
for r in clip_records:
    kept = _kept_boxes_at(r["frame_sampled"] * RUN_EVERY_N_FRAMES)
    iou = max((calculate_iou(r["bbox"], b) for b in kept), default=None)
    label = ("unlabelled (no kept-track mask here)" if iou is None else
             "on kept track" if iou >= CANDIDATE_LABEL_IOU else "not on kept track")
    cand_rows.append({**r, "label": label, "iou_with_kept": None if iou is None else round(float(iou), 3

pos = [r for r in cand_rows if r["label"] == "on kept track"]
neg = [r for r in cand_rows if r["label"] == "not on kept track"]
n_unlabelled = len(cand_rows) - len(pos) - len(neg)


def _stats(rows, key):
    v = [r[key] for r in rows]
    return f"min {min(v):7.2f}  median {float(np.median(v)):7.2f}  max {max(v):7.2f}" if v else "(none)"


candidate_summary = {"n_candidates": len(cand_rows), "n_on_kept_track": len(pos), "n_not_on_kept_track":
                     "n_unlabelled": n_unlabelled,
                     "accepted_on_kept_track": sum(r["accepted"] for r in pos),
                     "accepted_not_on_kept_track": sum(r["accepted"] for r in neg)}
print(f"{len(cand_rows)} candidates scored: {len(pos)} on the kept track, {len(neg)} not, {n_unlabelled}
print(f"Accepted at confidence > {CLIP_CONFIDENCE}: {candidate_summary['accepted_on_kept_track']} on the
      f"{candidate_summary['accepted_not_on_kept_track']} not on it (these are CLIP false positives, if
for name, rows in (("on kept track", pos), ("not on kept track", neg)):
    print(f"{name:<18} p_target  {_stats(rows, 'p_target')}")
    print(f"{'':<18} margin    {_stats(rows, 'margin')}")

if pos and neg:
    for key in ("p_target", "margin"):
        weakest_pos = min(r[key] for r in pos)
        still_in = sum(1 for r in neg if r[key] >= weakest_pos)
        gap = weakest_pos - max(r[key] for r in neg)
        candidate_summary[f"{key}_gap_pos_min_minus_neg_max"] = round(float(gap), 4)
        candidate_summary[f"{key}_negatives_scoring_at_least_weakest_positive"] = still_in
        print(f"\n[{key}] weakest on-track candidate = {weakest_pos:.3f}; {still_in} of {len(neg)} off-t
              f"score at least that high; gap (weakest positive - strongest negative) = {gap:.3f} "
              f"({'separable' if gap > 0 else 'overlapping'} on this clip)")
else:
    print("\nNeed both on-track and off-track candidates to compare scores.")

print("\nAll candidates, strongest margin first:")
print(f"{'frame':>5} {'box':>4}  {'label':<38}{'iou':>6}{'p_target':>10}{'margin':>9}  accepted")
for r in sorted(cand_rows, key=lambda r: -r["margin"]):
    iou_txt = "" if r["iou_with_kept"] is None else f"{r['iou_with_kept']:.2f}"
    print(f"{r['frame_sampled']:>5} {r['box_idx']:>4}  {r['label']:<38}{iou_txt:>6}{r['p_target']:>10.3f

with open(f"{ANALYSIS_DIR}/clip_candidates.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["frame_sampled", "box_idx", "label", "iou_with_kept", "p_targ
                                           "accepted", "bbox"])
    writer.writeheader()
    for r in cand_rows:
        writer.writerow({k: r[k] for k in writer.fieldnames})
with open(f"{ANALYSIS_DIR}/candidate_summary.json", "w") as f:
    json.dump(candidate_summary, f, indent=2)

if pos and neg:
    fig, axes = plt.subplots(1, 2, figsize=(10, 3.6))
    jitter = np.random.default_rng(0)
    for ax, key, title in ((axes[0], "p_target", "CLIP probability of target"),
                           (axes[1], "margin", "Logit margin (target - strongest other)")):
        for g, (rows, colour) in enumerate(((pos, "tab:green"), (neg, "tab:red"))):
            ax.scatter(g + jitter.uniform(-0.12, 0.12, len(rows)), [r[key] for r in rows], c=colour, alp
        ax.set_xticks([0, 1])
        ax.set_xticklabels(["on kept track", "not on kept track"])
```

```
        ax.set_title(title, fontsize=10)
    axes[0].axhline(CLIP_CONFIDENCE, color="k", linestyle="--", linewidth=0.8)
    plt.tight_layout()
    plt.savefig(f"{ANALYSIS_DIR}/clip_candidates.png", dpi=120)
    plt.show()
```

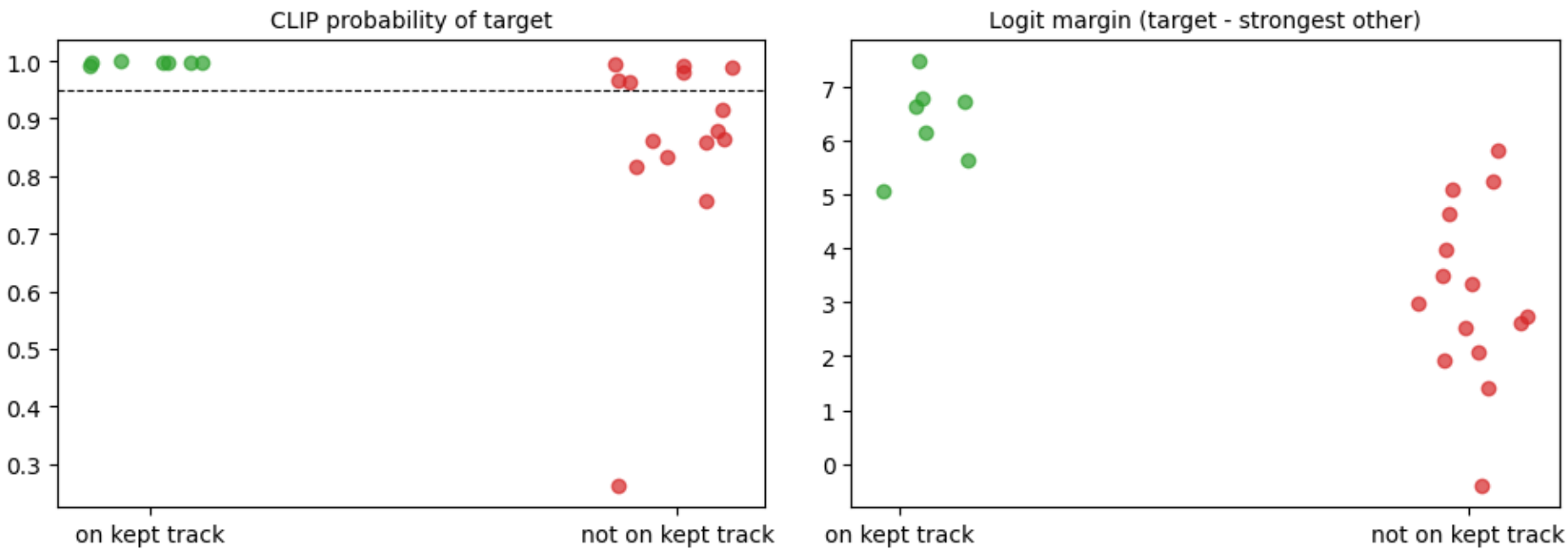
```
22 candidates scored: 7 on the kept track, 15 not, 0 unlabelled
Accepted at confidence > 0.95: 7 on the kept track, 6 not on it (these are CLIP false positives, if KEEP_

on kept track      p_target  min    0.99  median    1.00  max    1.00
                   margin    min    5.07  median    6.64  max    7.48
not on kept track  p_target  min    0.26  median    0.88  max    0.99
                   margin    min   -0.41  median    2.99  max    5.82

[p_target] weakest on-track candidate = 0.992; 2 of 15 off-track candidates score at least that high; gap

[margin] weakest on-track candidate = 5.066; 3 of 15 off-track candidates score at least that high; gap (

All candidates, strongest margin first:
frame  box  label                                 iou  p_target   margin  accepted
    1    0  on kept track                       0.98     0.999     7.48  True
    0    0  on kept track                       0.60     0.998     6.80  True
    6    0  on kept track                       0.96     0.999     6.74  True
    2    0  on kept track                       0.99     0.998     6.64  True
    7    0  on kept track                       0.96     0.998     6.15  True
    4    3  not on kept track                   0.00     0.994     5.82  True
    4    0  on kept track                       0.95     0.996     5.63  True
    3    2  not on kept track                   0.00     0.993     5.25  True
    4    1  not on kept track                   0.00     0.990     5.10  True
    3    0  on kept track                       0.98     0.992     5.07  True
    5    4  not on kept track                   0.00     0.980     4.65  True
    7    3  not on kept track                   0.00     0.965     3.98  True
    5    0  not on kept track                   0.48     0.967     3.50  True
    4    4  not on kept track                   0.00     0.915     3.34  False
    4    2  not on kept track                   0.00     0.879     2.99  False
    5    3  not on kept track                   0.00     0.864     2.75  False
    5    2  not on kept track                   0.00     0.816     2.61  False
    5    1  not on kept track                   0.00     0.859     2.53  False
    6    1  not on kept track                   0.00     0.863     2.09  False
    7    1  not on kept track                   0.00     0.834     1.92  False
    3    1  not on kept track                   0.00     0.757     1.42  False
    7    2  not on kept track                   0.00     0.261    -0.41  False
```



Save results to Drive

```
sync_run_to_drive("final")
print("\nFiles in", RUN_DRIVE)
for name in sorted(os.listdir(RUN_DRIVE)):
    print(f"  {name}  ({os.path.getsize(os.path.join(RUN_DRIVE, name)) / 1e6:.2f} MB)")
print("\nTimings (s):", timings)
```

```
[sync:final] saved to /content/drive/MyDrive/DS340WCourseProject/runs/analysis_dev: {'video_segments.zip'

Files in /content/drive/MyDrive/DS340WCourseProject/runs/analysis_dev
  analysis  (0.00 MB)
  keep_boxes.txt  (0.00 MB)
  run_info.json  (0.00 MB)
  tracked_video.mp4  (11.43 MB)
  video_segments.zip  (0.26 MB)
  video_segments_all_tracks.zip  (0.42 MB)
  video_segments_pruned.zip  (0.26 MB)

Timings (s): {'extract_frames': 3.29, 'mask_generation': 46.89, 'clip_filtering': 7.47, 'propagation': 17
```